Compolsory Assignment 4: Variational Autoencoders

Please fill out the the group name, number, members and optionally the name below.

Group number: 22

Group member 1: Oliver Tschudi Madsen

Group member 2: Tobias Tessem Hole

Group member 3: Mehdi Jamali

Group name (optional): Swaggity Swag

Assignment Submission

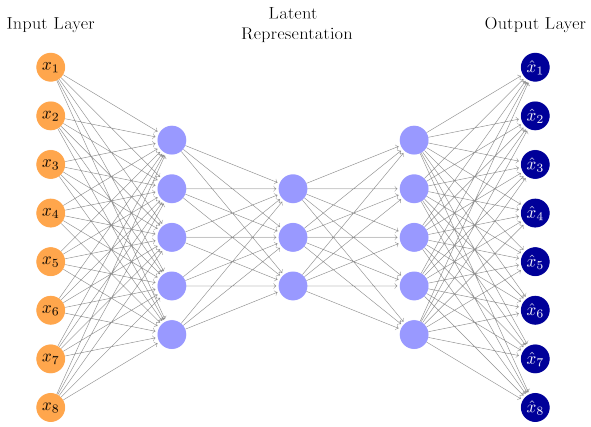
To complete this assignment answer the relevant questions in this notebook and write the code required to implement the relevant models. It concist of **two tasks**. At the end of each task, there are discussion questions that *must* be answered. The assignemnt is submitted by handing in this notebook as an .ipynb file and as a .pdf file.

We will explore autoencoder networks using the **Sign Language MNIST** dataset from Kaggle (as same as CA1). It contains 28×28 grayscale images of hand signs representing letters A–Y (excluding J and Z), for 24 classes in total. The data is provided as CSV files with a label column and 784 pixel columns. The standard split includes 27,455 training images and 7,172 test images

Autoencoder is an introduction to encoder-decoder architecture. Here we will use an encoder to transform the data into an latent representation, and a decoder to project it back into the visual space. Two main components: an **encoder** and a **decoder**.

- The encoder takes in the input data and maps it to a lower-dimensional latent representation. It typically consists of multiple layers that gradually reduce the dimensionality of the input data, capturing its essential features. The output of the encoder is a compressed representation of the input, often referred to as a code or latent vector.
- The decoder, on the other hand, takes the code from the encoder and

reconstructs the original data from it. It mirrors the architecture of the encoder by gradually expanding the code back to the original dimensionality. The output of the decoder is a reconstruction of the input data, which ideally should closely resemble the original input.



Task 1

- Load the SignMNIST dataset.
    - Use the provided CSV files sign_mnist_train.csv and sign_mnist_test.csv.
    - Normalize the images to have pixel values between 0 and 1.
- Build an autoencoder model.
    - The encoder should compress the images into a lower-dimensional latent space.
    - The decoder should reconstruct the images from the latent space.
    - The intermediat layers can either be linear or convolutional.
- Train the autoencoder.
    - Use `binary-cross-entropy` loss function for *reconstruction* (i.e. target value = input value)
    - You can choose optimizer freely (Adam is recommended).
- Visualize the reconstructed images.
    - *Compare* original and reconstructed images.
- Visualize the latent space
    - Use PCA to visualize the 2d-latent space.

    - Use the digit as labels
- Answer the discussion questions

In [3]:
```python
# Import necessary libraries

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import os
import struct
import seaborn as sns
import tensorflow as tf
from tensorflow.keras import layers, models
```

In [4]:
```python
# 1. Load and preprocess the Sign MNIST dataset

train = pd.read_csv('sign_mnist_train.csv')
test = pd.read_csv('sign_mnist_test.csv')

X_train = train.drop(columns=['label']).to_numpy(dtype=np.float32)
y_train = train['label'].to_numpy(dtype=np.int32)
X_test  = test.drop(columns=['label']).to_numpy(dtype=np.float32)
y_test  = test['label'].to_numpy(dtype=np.int32)

X_train = X_train.reshape(-1,28,28,1) / 255.0
X_test  = X_test.reshape(-1,28,28,1) / 255.0
```

In [5]:
```python
# 2. Build the autoencoder model

def build_encoder(latent_dim=64):
    encoder = models.Sequential([
        layers.Input(shape=(28, 28, 1)),
        layers.Conv2D(32, (3, 3), activation='relu', padding='same'),
        layers.MaxPooling2D((2, 2), padding='same'),
        layers.Conv2D(64, (3, 3), activation='relu', padding='same'),
        layers.MaxPooling2D((2, 2), padding='same'),
        layers.Conv2D(128, (3, 3), activation='relu', padding='same'),
        layers.Flatten(),
        layers.Dense(latent_dim, activation='relu', name='latent_vector')
    ])
    return encoder

def build_decoder(latent_dim=64):
    decoder = models.Sequential([
        layers.Input(shape=(latent_dim,)),
        layers.Dense(7 * 7 * 128, activation='relu'),
        layers.Reshape((7, 7, 128)),
        layers.Conv2DTranspose(64, (3, 3), strides=2, activation='relu', pad
        layers.Conv2DTranspose(32, (3, 3), strides=2, activation='relu', pad
        layers.Conv2DTranspose(1, (3, 3), activation='sigmoid', padding='same
    ])
```

```
    return decoder

def build_autoencoder(latent_dim=64):
    encoder = build_encoder(latent_dim)
    decoder = build_decoder(latent_dim)

    input_img = layers.Input(shape=(28,28,1))
    encoded  = encoder(input_img)
    decoded = decoder(encoded)

    autoencoder = models.Model(inputs=input_img, outputs=decoded, name="Convo
    autoencoder.compile(optimizer='adam', loss='binary_crossentropy')
    return autoencoder, encoder, decoder
```

In [6]:
```
autoencoder, encoder, decoder = build_autoencoder(latent_dim=64)
autoencoder.summary()

# Call fit with a single (x, y) pair — previous code passed duplicate positio
# causing "multiple values for argument 'epochs'".
history = autoencoder.fit(
    X_train, X_train,
    epochs=20,
    batch_size=128,
    shuffle=True,
    validation_data=(X_test, X_test)
)

plt.plot(history.history['loss'], label='Training Loss')
plt.plot(history.history['val_loss'], label='Validation Loss')
plt.title('Autoencoder Training Loss')
plt.xlabel('Epochs')
plt.ylabel('Loss')
plt.legend()
plt.show()
```

Model: "Convolutional_Autoencoder"

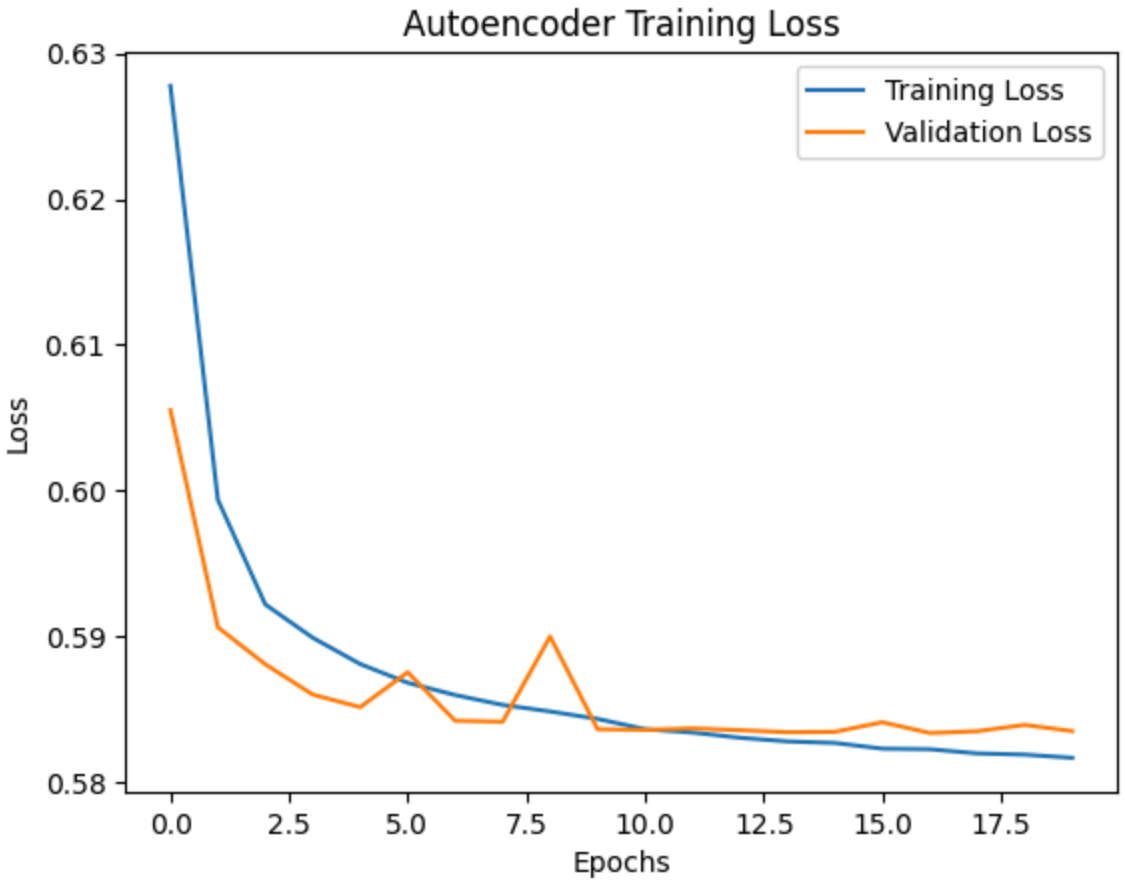
Layer (type)	Output Shape	Param
input_layer_2 (InputLayer)	(None, 28, 28, 1)	
sequential (Sequential)	(None, 64)	494,14
sequential_1 (Sequential)	(None, 28, 28, 1)	500,22

Total params: 994,369 (3.79 MB)

Trainable params: 994,369 (3.79 MB)

Non-trainable params: 0 (0.00 B)

```
Epoch 1/20
215/215 ━━━━━━━━━━━━━━━━━━━━ 42s 168ms/step - loss: 0.6278 - val_loss: 0.6055
Epoch 2/20
215/215 ━━━━━━━━━━━━━━━━━━━━ 35s 161ms/step - loss: 0.5994 - val_loss: 0.5906
Epoch 3/20
215/215 ━━━━━━━━━━━━━━━━━━━━ 42s 165ms/step - loss: 0.5922 - val_loss: 0.5881
Epoch 4/20
215/215 ━━━━━━━━━━━━━━━━━━━━ 46s 187ms/step - loss: 0.5899 - val_loss: 0.5860
Epoch 5/20
215/215 ━━━━━━━━━━━━━━━━━━━━ 43s 201ms/step - loss: 0.5881 - val_loss: 0.5851
Epoch 6/20
215/215 ━━━━━━━━━━━━━━━━━━━━ 43s 199ms/step - loss: 0.5868 - val_loss: 0.5875
Epoch 7/20
215/215 ━━━━━━━━━━━━━━━━━━━━ 42s 195ms/step - loss: 0.5860 - val_loss: 0.5842
Epoch 8/20
215/215 ━━━━━━━━━━━━━━━━━━━━ 40s 186ms/step - loss: 0.5853 - val_loss: 0.5841
Epoch 9/20
215/215 ━━━━━━━━━━━━━━━━━━━━ 40s 185ms/step - loss: 0.5848 - val_loss: 0.5900
Epoch 10/20
215/215 ━━━━━━━━━━━━━━━━━━━━ 38s 171ms/step - loss: 0.5843 - val_loss: 0.5836
Epoch 11/20
215/215 ━━━━━━━━━━━━━━━━━━━━ 36s 169ms/step - loss: 0.5836 - val_loss: 0.5836
Epoch 12/20
215/215 ━━━━━━━━━━━━━━━━━━━━ 35s 164ms/step - loss: 0.5834 - val_loss: 0.5837
Epoch 13/20
215/215 ━━━━━━━━━━━━━━━━━━━━ 33s 155ms/step - loss: 0.5830 - val_loss: 0.5836
Epoch 14/20
215/215 ━━━━━━━━━━━━━━━━━━━━ 69s 320ms/step - loss: 0.5828 - val_loss: 0.5834
Epoch 15/20
215/215 ━━━━━━━━━━━━━━━━━━━━ 36s 166ms/step - loss: 0.5827 - val_loss: 0.5834
Epoch 16/20
215/215 ━━━━━━━━━━━━━━━━━━━━ 35s 164ms/step - loss: 0.5823 - val_loss: 0.5841
Epoch 17/20
215/215 ━━━━━━━━━━━━━━━━━━━━ 36s 166ms/step - loss: 0.5822 - val_loss: 0.5834
Epoch 18/20
215/215 ━━━━━━━━━━━━━━━━━━━━ 36s 166ms/step - loss: 0.5820 - val_loss: 0.5835
Epoch 19/20
215/215 ━━━━━━━━━━━━━━━━━━━━ 39s 181ms/step - loss: 0.5819 - val_loss: 0.5839
Epoch 20/20
215/215 ━━━━━━━━━━━━━━━━━━━━ 36s 167ms/step - loss: 0.5817 - val_loss: 0.5835
```



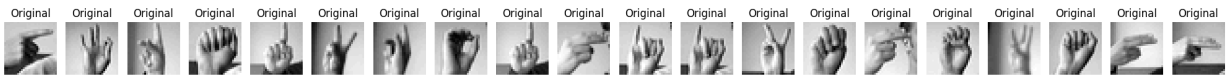
In [7]:
```python
# 4. Visualize the reconstructed images
n_images = 20
decoded_imgs = autoencoder.predict(X_test[:n_images])

plt.figure(figsize=(20, 4))
for i in range(n_images):

    ax = plt.subplot(2, n_images, i + 1)
    plt.imshow(X_test[i].reshape(28, 28), cmap='gray')
    plt.title("Original")
    plt.axis("Off")

plt.tight_layout()
plt.show()
```

```
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 296ms/step
```



In [8]:
```python
# 4. Visualize the PCA-projection of the latent space. Try to inlcude the lab
# the distribution of the latent space.

from sklearn.decomposition import PCA

latent_representations = encoder.predict(X_test)
```

```
pca = PCA(n_components=2)
latent_2d = pca.fit_transform(latent_representations)

plt.figure(figsize=(10, 8))
scatter = plt.scatter(
    latent_2d[:, 0],
    latent_2d[:, 1],
    c=y_test,
    cmap='tab10',
    alpha=0.7,
    s=15
)

plt.colorbar(scatter, label='Class Label')
plt.title('PCA Projection Of The Latent Space')
plt.xlabel('Principal Component 1')
plt.ylabel('Principal Component 2')
plt.grid(True, linestyle='--', alpha=0.5)
plt.show()
```

```
225/225 ━━━━━━━━━━━━━━━━━━━━ 2s 10ms/step
```



Discussion

1. Why do we normalize the pixel values of the images?
2. How does the size of the latent space affect the reconstructed images?
3. Can autoencoders be used for noise reduction? How?

Discuss here

1

The "unnormalized" images tend to be quuite big. By normalizing them to a smaller sccale like [0,1] they are smaller and easier to work with. The "Adam" optimizer usually works best with these normalized images. The same goes for the Relu and sigmoiid function as they tend to work more efficiently with smaller inputs and achieves a better gradient flow. It also improves the generalization of

the model, and reduces variaton in the images.

2

The size of the latent space can improve several points in the image reconstruction. If the space is larger, the reconstruction can imporve. If the space is big enough the encoder may store more details from the input and the decoder has more information availabe to create better features. This is vital as the reconcstructed images can be displayed better than the original images.

If the latent space is smaller, the quality of the images may weaken. The details fade and the reconstructed images loses sharpness and its good qualities. The encoder and the decoder does not have the same vital information compared to the bigger latent space. If the latent space is too small the data will likely underfit as there is not enough information to process.

3

Yes, an autoencoder can be used to reduce noise. If the autoencoder is trained to reconstructed to clean images from noisy outputs. By training the autoencoder to just reconstruct original images. Then the trained model is used to denoise new images. This way the encoder learns to ignore noise, and decoder learns to rebuild new clean images.

Task 2

In contrast to simple autoencoders, which aim to learn a deterministic mapping from input data to a compressed latent space and back, **Variational Autoencoders (VAEs)** introduce a probabilistic approach to the latent space. While simple autoencoders can reconstruct data effectively, they are not inherently capable of generating new, realistic data samples. VAEs, on the other hand, are designed for *both reconstruction and generation* of novel data samples, making them highly useful for generative modeling tasks.

Why VAEs? A simple autoencoder maps input data to a fixed point in the latent space, meaning that each input has a single corresponding latent vector. While this approach works well for compression and reconstruction, it lacks the ability to generalize and generate new data points because the latent space is not continuous or well-structured.

In contrast, a Variational Autoencoder treats the latent space probabilistically.

Instead of mapping each input to a single point, it maps inputs to a distribution (typically a Gaussian distribution) in the latent space. This probabilistic treatment allows for the generation of new samples by sampling from the learned distribution, thus enabling the model to create *new*, unseen data points that resemble the training data.

How VAEs Work? In a VAE, the *encoder* maps the input data to two vectors: a *mean vector* and a *variance (or log variance) vector* (in contrast to one as in a traditional AE). These vectors representing a Gaussian distribution in the latent space. Instead of encoding to a fixed point, the model samples a latent vector from this distribution, using these mean and variance parameters. This sampling introduces variability, allowing the VAE to explore different parts of the latent space and generate diverse outputs.

The *decoder* then reconstructs the data by decoding from the sampled latent vector, which can either come from the encoded input or be randomly generated. This process makes VAEs ideal for generating novel data samples in applications like image generation.

The loss function? The loss function plays a crucial role in training the model to both reconstruct input data and ensure that the latent space is properly structured. The VAE loss function consists of two main components:

1. Reconstruction Loss

Measures how well the VAE is able to reconstruct the input data from the latent space.
Typically, this is computed using **binary cross-entropy** (for normalized image data) or **mean squared error** (for continuous data).
For images, binary cross-entropy is often used.

$$ \mathcal{L}_{\text{reconstruction}}(x, \hat{x}) = - \sum_{i=1}^{N} \left[x_i \log(\hat{x}_i) + (1 - x_i)\log(1 - \hat{x}_i) \right] $$

Where:

- x_i is the original data point (e.g., pixel value)
- $\hat{x}_i$ is the reconstructed data point
- N is the number of data points or pixels

---

2. KL Divergence Loss

Ensures that the learned latent distribution $q(z|x)$, parameterized by μ and σ^2,
is close to a standard normal distribution $p(z) = \mathcal{N}(0, I)$.

$$ \mathcal{L}_{\text{KL}}(q(z|x) \parallel p(z)) = \frac{1}{2} \sum_{j=1}^{d} \left(1 + \log(\sigma_j^2) - \mu_j^2 - \sigma_j^2 \right) $$

Where:

- μ_j is the mean of the latent variable distribution
- σ_j^2 is the variance of the latent variable distribution
- d is the dimensionality of the latent space

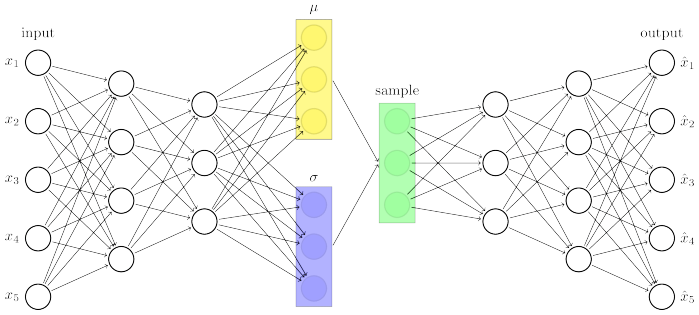
🧮 Total VAE Loss

The total loss function for a VAE is the sum of the reconstruction loss and the KL divergence loss:

$$ \mathcal{L}_{\text{VAE}} = \mathcal{L}_{\text{reconstruction}} + \mathcal{L}_{\text{KL}} $$

The Role of KL Divergence A crucial aspect of training VAEs is ensuring that the latent space is well-structured and that the learned distributions align with a *prior distribution* (typically a standard normal distribution). This is where the *Kullback-Leibler (KL) divergence loss* comes in. KL divergence measures how different the learned distribution is from the desired prior distribution. By minimizing KL divergence during training, the VAE ensures that the latent space follows a continuous, smooth distribution, making it easier to generate realistic new samples by sampling from this space.

In summary, VAEs combine the strengths of autoencoders for reconstruction with a probabilistic latent space, enabling the generation of new data samples and ensuring smooth interpolation within the latent space through the use of KL divergence. This combination makes VAEs a powerful generative model in unsupervised learning tasks.



Run the following code

In [1]:
```
import numpy as np
import pandas as pd
import tensorflow as tf
from tensorflow.keras import layers, Model, backend as K
import matplotlib.pyplot as plt
import keras
from keras import ops, layers
from tensorflow.keras.layers import Layer
```

In [9]:
```
# Load the MNIST dataset.
train_df = pd.read_csv("sign_mnist_train.csv")
test_df  = pd.read_csv("sign_mnist_test.csv")

x_train = train_df.drop(columns=['label']).to_numpy().astype('float32') / 2
x_test  = test_df.drop(columns=['label']).to_numpy().astype('float32') / 25

x_train = np.reshape(x_train, (len(x_train), 28, 28, 1))
x_test  = np.reshape(x_test,  (len(x_test), 28, 28, 1))

y_train = train_df['label'].to_numpy()
y_test  = test_df['label'].to_numpy()
```

In [44]:
```
latent_dim = 8  # Dimensionality of the latent space (small for simplicity)
input_shape = (28, 28, 1)  # Input image shape
```

In [45]:
```
# We define the encoder in a similar way as the autoencoder
input_img = layers.Input(shape=input_shape)
x = layers.Conv2D(32, 3, activation='relu', padding='same')(input_img)
x = layers.MaxPooling2D(2, padding='same')(x)
x = layers.Conv2D(64, 3, activation='relu', padding='same')(x)
x = layers.MaxPooling2D(2, padding='same')(x)
x = layers.Flatten()(x)
x = layers.Dense(128, activation='relu')(x)
```

In [47]:
```
# However, instead of a single output layer to the latent space, we have tw
z_mean = layers.Dense(latent_dim)(x)
z_log_var = layers.Dense(latent_dim)(x)
```

In [48]:
```
# We define a sampling function to sample from the latent space, which intr
```

```
# Sampling function
def sampling(args):
    z_mean, z_log_var = args
    epsilon = K.random_normal(shape=K.shape(z_mean))
    return z_mean + K.exp(0.5 * z_log_var) * epsilon
```

In [49]:
```
# There are multiple ways to sample from the latent space, here I introduce
# Note the output shape of the Lambda layer; we are returning the (sampled)
z = layers.Lambda(sampling, output_shape=(latent_dim,))([z_mean, z_log_var]
```

In [50]:
```
# The decoder is defined in the same way as the autoencoder
decoder_input = layers.Input(shape=(latent_dim,))
d = layers.Dense(7 * 7 * 64, activation='relu')(decoder_input)
d = layers.Reshape((7, 7, 64))(d)
d = layers.Conv2DTranspose(64, 3, activation='relu', strides=2, padding='sa
d = layers.Conv2DTranspose(32, 3, activation='relu', strides=2, padding='sa
decoded_img = layers.Conv2DTranspose(1, 3, activation='sigmoid', padding='s
```

In a Variational Autoencoder (VAE), adjusting the weight of the KL divergence loss controls the balance between data reconstruction accuracy and latent space regularization. Increasing the weight encourages a more structured and continuous latent space but may reduce reconstruction quality, while decreasing it improves reconstruction but risks losing meaningful structure in the latent space.

In [51]:
```
#KL divergence loss weight. This is a hyperparameter that can be tuned, run
# This is the value you should experiment with
KL_weight = 1.0
```

In [52]:
```
# Having build our layers, we can now define the models
encoder = Model(input_img, [z_mean, z_log_var, z], name='encoder')
decoder = Model(decoder_input, decoded_img, name='decoder')

# VAE model that connects the encoder and decoder
vae_output = decoder(encoder(input_img)[2])
vae = Model(input_img, vae_output, name='vae')
```

In [53]:
```
# VAE Loss Layer
# Variables: x (input), x_hat (reconstruction), mu (z_mean), logvar (z_log_

class VAE_LossLayer(Layer):
    def __init__(self, beta=0.1, rec_kind="bce", **kwargs):
        super(VAE_LossLayer, self).__init__(**kwargs)
        self.beta = beta
        self.rec_kind = rec_kind

    def call(self, inputs):
        x, x_hat, mu, logvar = inputs
```

```
        # Reconstruction loss
        if self.rec_kind == "bce":
            rec_loss = -tf.reduce_sum(
                x * tf.math.log(tf.clip_by_value(x_hat, 1e-7, 1.0)) +
                (1 - x) * tf.math.log(tf.clip_by_value(1 - x_hat, 1e-7, 1.0
                axis=[1, 2, 3]
            )
        elif self.rec_kind == "mse":
            rec_loss = tf.reduce_sum(tf.square(x - x_hat), axis=[1, 2, 3])
        else:
            raise ValueError("rec_kind must be 'bce' or 'mse'")

        # KL Divergence
        kl_loss = -0.5 * tf.reduce_sum(1 + logvar - tf.square(mu) - tf.exp(

        # Total loss
        total_loss = tf.reduce_mean(rec_loss + self.beta * kl_loss)

        self.add_loss(total_loss)
        return x_hat
```

In [54]:

```
#Train the VAE

lossed_output = VAE_LossLayer(beta=0.1, rec_kind="bce")(
    [input_img, vae_output, z_mean, z_log_var]
)

vae = keras.Model(inputs=input_img, outputs=lossed_output, name="vae")

vae.compile(optimizer="adam")

#Feel free to play around with the paramaters
history = vae.fit(
    x_train,
    epochs=25,
    batch_size=128,
    validation_data=(x_test,),
    verbose=2,
)
```

```
Epoch 1/25
215/215 - 37s - 170ms/step - loss: 499.9456 - val_loss: 484.1967
Epoch 2/25
215/215 - 32s - 147ms/step - loss: 482.5285 - val_loss: 474.4552
Epoch 3/25
215/215 - 32s - 147ms/step - loss: 475.3571 - val_loss: 471.4836
Epoch 4/25
215/215 - 31s - 146ms/step - loss: 472.9730 - val_loss: 470.4097
Epoch 5/25
215/215 - 32s - 147ms/step - loss: 471.5594 - val_loss: 470.4397
Epoch 6/25
215/215 - 267s - 1s/step - loss: 470.6346 - val_loss: 469.7906
Epoch 7/25
215/215 - 33s - 151ms/step - loss: 469.7546 - val_loss: 469.2824
Epoch 8/25
215/215 - 31s - 146ms/step - loss: 469.2163 - val_loss: 469.1530
Epoch 9/25
215/215 - 41s - 193ms/step - loss: 468.6705 - val_loss: 469.1889
Epoch 10/25
215/215 - 31s - 145ms/step - loss: 468.2252 - val_loss: 469.2368
Epoch 11/25
215/215 - 31s - 145ms/step - loss: 467.8991 - val_loss: 469.0898
Epoch 12/25
215/215 - 31s - 146ms/step - loss: 467.5653 - val_loss: 468.9136
Epoch 13/25
215/215 - 31s - 145ms/step - loss: 467.3179 - val_loss: 468.9956
Epoch 14/25
215/215 - 31s - 144ms/step - loss: 466.9994 - val_loss: 468.8174
Epoch 15/25
215/215 - 31s - 146ms/step - loss: 466.8911 - val_loss: 468.7127
Epoch 16/25
215/215 - 32s - 150ms/step - loss: 466.5619 - val_loss: 468.7122
Epoch 17/25
215/215 - 31s - 143ms/step - loss: 466.4143 - val_loss: 468.6448
Epoch 18/25
215/215 - 31s - 143ms/step - loss: 466.3673 - val_loss: 468.6154
Epoch 19/25
215/215 - 31s - 143ms/step - loss: 466.0511 - val_loss: 468.6937
Epoch 20/25
215/215 - 31s - 145ms/step - loss: 465.9607 - val_loss: 468.7027
Epoch 21/25
215/215 - 31s - 144ms/step - loss: 465.8581 - val_loss: 468.8295
Epoch 22/25
215/215 - 31s - 145ms/step - loss: 465.6766 - val_loss: 468.7422
Epoch 23/25
215/215 - 31s - 144ms/step - loss: 465.5439 - val_loss: 469.5883
Epoch 24/25
215/215 - 31s - 146ms/step - loss: 465.5410 - val_loss: 468.7131
Epoch 25/25
215/215 - 31s - 144ms/step - loss: 465.2904 - val_loss: 468.8805
```

In [55]:
```
# Run this visualisation code after training the VAE to answer the discussi
```

```
def plot_latent_space(decoder, n=15, figsize=15, scale=2.0):


    latent_shape = decoder.input_shape
    try:
        latent_dim = int(latent_shape[1])
    except Exception:
        latent_dim = int(latent_shape[-1])

    figure = np.zeros((28 * n, 28 * n))
    grid_x = np.linspace(-scale, scale, n)
    grid_y = np.linspace(-scale, scale, n)

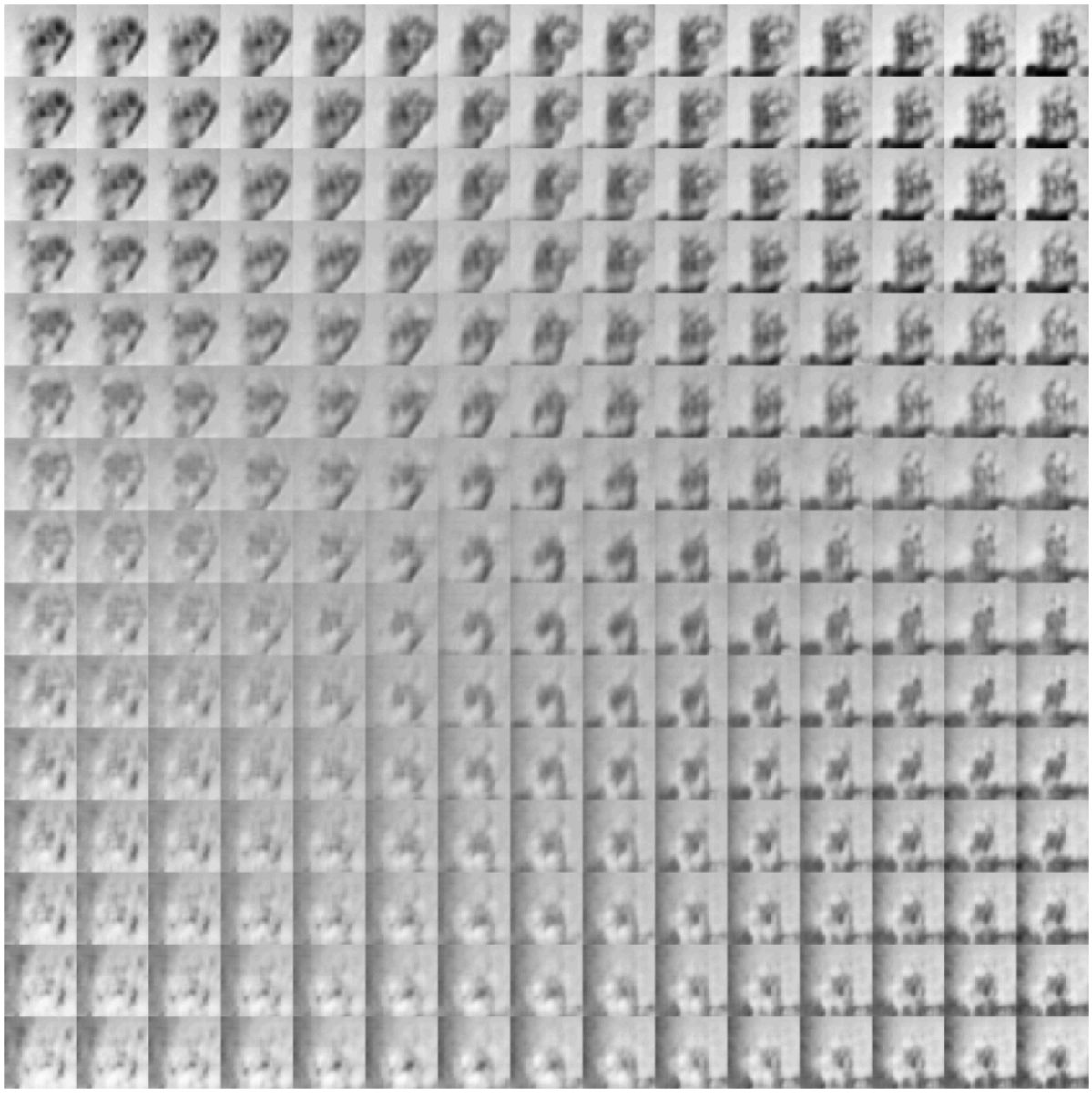
    for i, yi in enumerate(grid_y):
        for j, xi in enumerate(grid_x):

            if latent_dim == 2:
                z_sample = np.array([[xi, yi]], dtype=np.float32)
            else:
                z_sample = np.zeros((1, latent_dim), dtype=np.float32)
                z_sample[0, 0] = xi
                z_sample[0, 1] = yi

            x_decoded = decoder.predict(z_sample, verbose=0)
            digit = x_decoded[0].reshape(28, 28)
            figure[i * 28: (i + 1) * 28, j * 28: (j + 1) * 28] = digit

    plt.figure(figsize=(figsize, figsize))
    plt.imshow(figure, cmap='Greys_r')
    plt.axis('off')
    plt.show()

# Plot latent space
plot_latent_space(decoder)
```



🧪 Experiment Instructions

Tips: Refer to the visualization above and the loss output from your training cell (make sure `verbose` is set to 1 or 2) to answer the questions.

1. **Adjust the weight of the KL divergence term** (`beta`) in the VAE loss function (e.g., set it to **0.1**, **1**, and **5**).
   Run the model with each weight setting and observe the results.

2. **Change the dimensionality of the latent space** (`latent_dim`) to explore how it affects the model's behavior.
   Try setting `latent_dim` to **2**, **4**, and **8**.

Re-run your model for each setting and visualize:

- The reconstructed images
- The latent space projection (for 2D, plot directly; for higher dimensions, use PCA or t-SNE)

3. **Discussion:**
   Based on your observations:

   - How does changing the KL weight (`beta`) affect the balance between reconstruction quality and latent space smoothness?
   - How does increasing the latent space dimensionality (`latent_dim`) affect the expressiveness of the model and the clarity of the reconstructions?
   - Were these effects consistent with your expectations? Why or why not?

3

When the beta-value was set to 1, the images turned out rather blurry. It is possible to see the different signs on the hands, but there are not many distinguishing features that the naked eye can spot. The reconstruiction is decent, but the value can be changed further to get a better reconstruction.

Changing the the beta-value to 0.1, the reconstructed images are somewhat clearer. The images of the hands are a bit clearer than the a beta-value of 1. However, the loss values are rather similar.

For a higher beta-value of 5, the loss value is a bit higher and the picture quality drops greatly. Some of the hand gestures are almost unrecognizable. For the next task, we will use a low beta-value to maintain the overall quality.

Changing the latent space dimensionality did not do much for the quality. With the amount of dimensions doubled, the KL has more to work with. If the latent space is changed, the beta-value may also be a bit tuned to compensate for the increasing amount of dimensions. However, changing the dimensions to 8, did improve the quality a bit, but some distinguishing features were lost, for example the darker background in some of the images.

We were expecting some better images by tuning the dimensions, but this did not turn out quite as expected. This was plotted directly but could likely be visualized better with PCA or t-SNE.